

# Apunte de Swing — CardLayout, JTable y JOptionPane

*Cambiar de pantalla · mostrar datos en una tabla · avisar y confirmar*

Con estos tres componentes se arma la base de cualquier app de gestión: una pantalla con una tabla de datos, un formulario para cargar uno nuevo, y avisos para corregir o confirmar. El ejemplo de este apunte es una tabla de animales de la reserva.

## CardLayout: varias pantallas en una ventana

---

Un CardLayout apila varios paneles en el mismo lugar, como un mazo de cartas: solo se ve el de arriba. Sirve para simular pantallas distintas (la tabla, el formulario) sin abrir ventanas nuevas.

### Cómo se arma en NetBeans

- Un JPanel contenedor con layout **CardLayout** (clic derecho → Set Layout → Card Layout). En el ejemplo se llama panelCardL.
- Adentro, un JPanel por cada pantalla (panelTabla, panelForm). Cada uno se diseña por separado, con sus propios componentes.
- Al meter un panel adentro, NetBeans genera solo una línea que lo anota en el mazo con una etiqueta: `panelCardL.add(panelForm, "card3");`

### Un panel, dos nombres

Cada panel del mazo tiene dos nombres, y sirven para cosas distintas:

- Nombre de variable (**panelForm**): el que le ponés vos. Sirve para nombrar al panel en el código.
- Nombre de card ("**card3**"): la etiqueta con la que el mazo lo tiene anotado. La genera NetBeans. Sirve para pedirle al mazo que lo muestre.

Es como una compañera que vos conocés como Ana, pero en secretaría figura con el legajo 3. Si pedís “traeme a Ana”, secretaría no entiende, porque solo trabaja con legajos. El CardLayout es la secretaría: solo entiende el nombre de card.

### Cambiar de card

Se hace desde el evento de un botón: se le pide al CardLayout que muestre la etiqueta que se quiere.

```
CardLayout cl = (CardLayout) panelCardL.getLayout();  
cl.show(panelCardL, "card3"); // muestra el panel anotado como card3
```

→ El texto entre comillas tiene que ser igual al del add(). Si no coincide no da error: simplemente no cambia nada.

→ `getLayout()` devuelve el layout como tipo general, por eso hay que castearlo a CardLayout. También existen `cl.next(panelCardL)`, `cl.previous(panelCardL)`, `cl.first(...)` y `cl.last(...)`.

### Qué botón lleva adónde

En el ejemplo: **AGREGAR** (en la tabla) muestra card3, el formulario. **GUARDAR** y **CANCELAR** (en el formulario) muestran card2, la tabla. La card que se ve al abrir la ventana es la primera que se agregó.

→ Un botón que cambia de pantalla tiene que estar dentro de la card desde la que el usuario lo va a usar.

## JTable y DefaultTableModel

---

Un JTable muestra datos en filas y columnas. La tabla en sí no guarda nada: los datos viven en un modelo, el DefaultTableModel, y la tabla se dibuja a partir de él. Por eso, para agregar o sacar filas se trabaja con el modelo.

## Armarla en NetBeans

Se arrastra **Table** desde la paleta (Swing Controls); NetBeans la envuelve solo en un JScrollPane. Con clic derecho → **Table Contents** se definen las columnas y el tipo de cada una (String, Integer, Boolean...).

→ El tipo de la columna importa: una columna Boolean se dibuja como un casillero.

## Obtener el modelo

Se guarda en un atributo de la clase, así se usa en todos los eventos:

```
private DefaultTableModel modelo;    // atributo de la clase

// en el constructor, después de initComponents():
modelo = (DefaultTableModel) tblAnimales.getModel();
modelo.setRowCount(0);    // borra las filas vacías que trae la tabla de fábrica
```

## Agregar, leer y eliminar filas

```
// agrega una fila
modelo.addRow(new Object[]{"Lobo", "Fenrir", 3, true, false});
// fila elegida (-1 si no hay ninguna)
int fila = tblAnimales.getSelectedRow();
// valor de una celda: fila elegida, columna 1
String nombre = (String) tblAnimales.getValueAt(fila, 1);
// elimina esa fila
modelo.removeRow(fila);
```

→ Siempre verificar que `fila != -1` antes de usarla: `removeRow(-1)` da error.

→ Los tipos tienen que coincidir con los de las columnas: en una columna Integer va un número, en una Boolean un `true/false`. Pasar un String en una columna Boolean puede dar `ClassCastException` al dibujar la tabla.

## JOptionPane: avisos y confirmaciones

---

Abre una ventanita estándar con una sola línea de código.

### showMessageDialog — avisar

```
JOptionPane.showMessageDialog(this,
    "Animal agregado correctamente.",    // mensaje
    "Éxito",                            // título de la ventana
    JOptionPane.INFORMATION_MESSAGE);    // ícono
```

Íconos: **INFORMATION\_MESSAGE** (aviso o éxito), **WARNING\_MESSAGE** (falta algo), **ERROR\_MESSAGE** (algo salió mal), **QUESTION\_MESSAGE** (pregunta).

### showConfirmDialog — preguntar

```
int respuesta = JOptionPane.showConfirmDialog(this,
    "¿Eliminar a Fenrir?", "Confirmar eliminación",
    JOptionPane.YES_NO_OPTION);

if (respuesta == JOptionPane.YES_OPTION) {
    modelo.removeRow(fila);
}
```

→ Devuelve un número; las constantes `YES_OPTION` y `NO_OPTION` evitan tener que recordar cuál es cuál.

→ `showInputDialog(this, "mensaje")` pide un texto y lo devuelve como `String` (o `null` si el usuario cancela).

## Radio y checkbox del formulario

---

El formulario del ejemplo usa dos componentes más:

- **JRadioButton + ButtonGroup**: los radios de un mismo grupo son excluyentes, solo uno puede estar marcado. En el diseñador se arrastra un `ButtonGroup` de la paleta y en cada radio se lo elige en `Properties` → `buttonGroup`.
- **JCheckBox**: un casillero que se marca o desmarca de forma independiente.

```
rdSano.isSelected()           // true si ese radio está marcado
rbgEstado.getSelection() == null // true si no se eligió ningún radio
rbgEstado.clearSelection();    // desmarca todos los radios del grupo
chkTratamiento.isSelected()   // true si el checkbox está marcado
```

## Todo junto: guardar un animal nuevo

---

El evento de GUARDAR combina los tres temas: avisa con `JOptionPane` si falta algo, agrega la fila al modelo y vuelve a la tabla con `CardLayout`.

```
if (nombre.isEmpty() || especie.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Faltan datos.", "Datos incompletos",
                                  JOptionPane.WARNING_MESSAGE);
    return; // corta acá: no sigue si falta algo
}
```

```
modelo.addRow(new Object[]{especie, nombre, edad,
                             rdSano.isSelected(), chkTratamiento.isSelected()});
```

```
CardLayout cl = (CardLayout) panelCardL.getLayout();
cl.show(panelCardL, "card2"); // vuelve a la pantalla de la tabla
```

→ Es una versión resumida: el evento completo también convierte la edad con `Integer.parseInt` dentro de un `try-catch`, para avisar si no escribieron un número.

## Para tu caja de herramientas

---

Estos componentes no son obligatorios en cualquier proyecto; se consultan cuando hacen falta:

- **JDialog**: una ventana secundaria con diseño propio, para formularios más complejos que un `JOptionPane`.
- **JList**: una lista de ítems siempre visibles, con selección simple o múltiple.
- **JMenuBar**: la barra de menú de arriba (Archivo, Editar...).
- **JFormattedTextField**: un campo que solo acepta un formato (fecha, número).

→ Los cuatro están explicados en el apunte de *Swing avanzado*.